

LAB-2: 内存管理初步

【注：本文件是lab2的原README文件，由于过于详细，太像实验报告而改掉了，由于里面也包含很多我们自己写的函数和原理解释，以后看起来说不定可以帮助理解，便决定以此文件存档】

在lab-2中, 我们开始认识和管理“程序除了CPU外最常访问的共享资源——内存”。

内存管理的实现不是一步到位的, lab-2中我们实现了**物理内存和内核态虚拟内存的管理**, 剩余部分将在后面的实验逐渐完善。

我们的分工如下：

丁熙妍：完成了**物理内存**部分，以及对应的实验文档。

吴晨曦：完成了**虚拟内存**部分，以及对应的实验文档。

代码组织结构

```
ECNU-OSLAB-2025-TASK
├─ LICENSE          开源协议
├─ .vscode          配置了可视化调试环境
├─ registers.xml    配置了可视化调试环境
├─ common.mk        Makefile中一些工具链的定义
├─ Makefile         编译运行整个项目（CHANGE，增加trap和mem目录作为target）
├─ kernel.ld        定义了内核程序在链接时的布局（CHANGE，增加一些关键位置的标记）
├─ pictures         README使用的图片目录（CHANGE，日常更新）
├─ README.md        实验指导书（CHANGE，日常更新）
├─ src              源码
│   └─ kernel       内核源码
│       └─ arch      RISC-V相关
│           └─ method.h
│           └─ mod.h
│           └─ type.h
│       └─ boot      机器启动
│           └─ entry.S
│           └─ start.c
│       └─ lock      锁机制
│           └─ spinlock.c
│           └─ method.h
│           └─ mod.h
│           └─ type.h
│       └─ lib       常用库
│           └─ cpu.c
│           └─ print.c
│           └─ uart.c
│           └─ utils.c（NEW，工具函数）
│           └─ method.h（CHANGE，utils.c的函数声明）
│           └─ mod.h
│           └─ type.h
│       └─ mem       内存模块
│           └─ pmem.c（本实验完成，物理内存管理）
│           └─ kvm.c（本实验完成，内核态虚拟内存管理）
│           └─ method.h（NEW）
│           └─ mod.h（NEW）
```

```

|   └─ type.h (NEW)
├─ trap  陷阱模块
|   └─ method.h (NEW)
|       └─ mod.h (NEW)
|           └─ type.h (NEW, 增加CLINT和PLIC寄存器定义)
└─ main.c (本实验完成)

```

第一阶段: 物理内存

1.1 物理内存布局

首先需要关注的文件是 `kernel.ld` 文件, 它规定了内核文件 `kernel-qemu.elf` 在载入内存时的布局

物理内存按照地址空间划分为三个部分:

- `0x80000000 ~ KERNEL_DATA` 存放了 `kernel-qemu.elf` 的代码
- `KERNEL_DATA ~ ALLOC_BEGIN` 存放了 `kernel-qemu.elf` 的数据
- `ALLOC_BEGIN ~ ALLOC_END` 属于 **未使用的可分配的物理页**

前两个区域的物理页会一直被内核占用, 不会纳入动态分配和回收的范围, 需要管理的只有第三个区域的物理页。

1.2 基本原理

物理内存管理的基本原理: **4KB物理页切分 + 空闲链表组织**

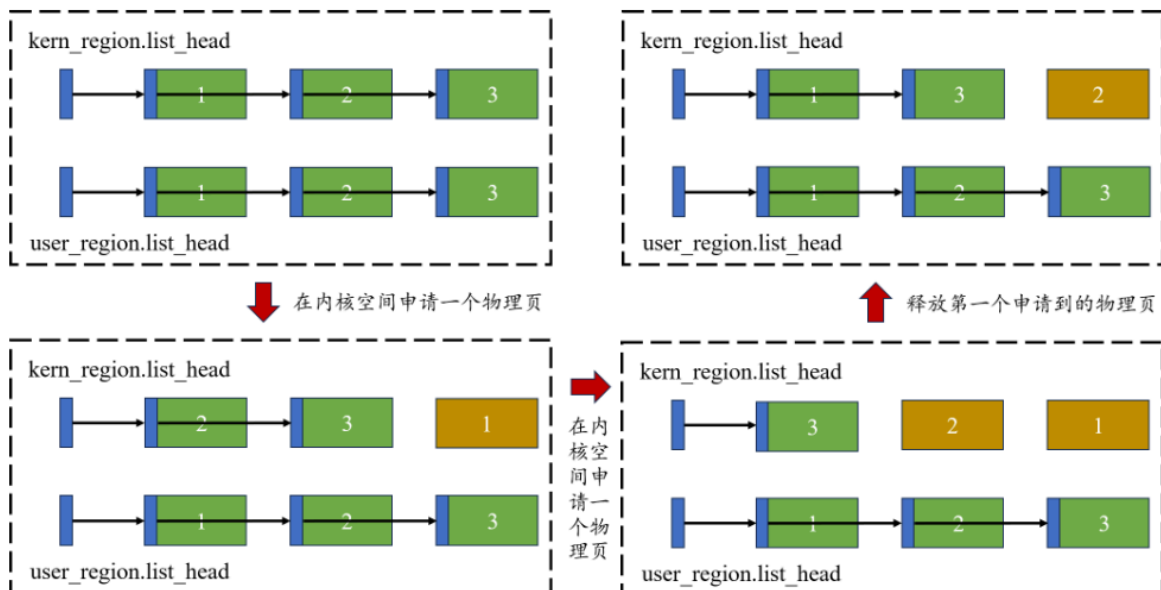
`ALLOC_BEGIN ~ ALLOC_END` 这块物理空间被切分为N个4KB物理页(不会有剩余)

此外, 考虑到内核与用户空间的强制隔离, 我们设置了两个 `alloc_region`, 基于 `KERNEL_PAGE` 进行边界划分:

`kernel_region` 记录了内核空间的空闲物理页情况, `user_region` 记录了用户空间的空闲物理页情况

`alloc_region` 描述了一组空闲页链表, 包括起止位置、空闲页面数量、链表头节点、保证一致性的锁

下图显示了物理页的申请和释放在链表上是如何体现的:



1.3 函数实现

物理内存管理的函数主要在 `pmem.c` 中实现，其中使用了 `utils.c` 中的一些辅助函数。

1.3.1 pmem_init: 初始化物理内存

初始化两段可分配物理内存区域（内核区与用户区），包括基本数值和空闲链表：

- 初始化 `kern_region` 和 `user_region` 的基本数值：`begin`，`end`，`allocable`，`list_head`，`lk`
- 使用 `pmem_free` 将两段区域的所有物理页加入各自的空闲链表

```
// 物理内存的初始化
// 本质上就是填写kern_region和user_region，包括基本数值和空闲链表
void pmem_init(void)
{
    // 初始化kern_region和user_region的锁
    spinlock_init(&kern_region.lk, "kernel_region_lock");
    spinlock_init(&user_region.lk, "user_region_lock");

    // 划分内核和用户内存区域的边界
    uint64 boundary = (uint64)ALLOC_BEGIN + KERN_PAGES * PGSIZE;

    // 初始化kern_region剩余内容
    kern_region.begin = (uint64)ALLOC_BEGIN;
    kern_region.end = boundary;
    kern_region.allocable = 0; // 可分配的空闲页面数
    kern_region.list_head.next = NULL; // 可分配链的链头节点

    // 初始化user_region剩余内容
    user_region.begin = boundary;
    user_region.end = (uint64)ALLOC_END;
    user_region.allocable = 0;
    user_region.list_head.next = NULL;

    // 将内核区域的物理页加入空闲链表
    // 相当于把内核区域的物理页都free掉
    for (uint64 p = kern_region.begin; p < kern_region.end; p += PGSIZE)
    {
        pmem_free(p, true);
    }
    // 将用户区域的物理页加入空闲链表
    for (uint64 p = user_region.begin; p < user_region.end; p += PGSIZE)
    {
        pmem_free(p, false);
    }
}
```

1.3.2 pmem_alloc: 申请空闲页面

申请一个4KB的物理页：

- 根据 `is_kernel` 选择对应的 `alloc_region`

- 获取 `alloc_region` 的锁, 从链表头取出第一个节点 (**头删法**) ; 更新链表头和 `allocable` , 释放锁
- 若无可用页, 则调用 `panic` 锁死
- 若申请成功, 则使用 `memset` 将该页**清零**, 并返回该页的地址

```
// 尝试返回一个可分配的清零后的物理页
// 失败则panic锁死
void* pmem_alloc(bool in_kernel)
{
    // 申请一个空闲页面:
    // 取出空闲链表的第一个空闲页作为分配的页面, 并清零后返回

    page_node_t *page;

    // 分配区域: 内核区域 or 用户区域
    alloc_region_t *ar = &user_region;
    if (in_kernel)
    {
        ar = &kern_region;
    }

    // 取出空闲链表的第一个节点
    spinlock_acquire(&ar->lk);
    page = ar->list_head.next;
    if (page) {
        ar->list_head.next = page->next;
        ar->allocable--;
    }
    spinlock_release(&ar->lk);

    // 分配失败, 则panic锁死
    if (!page) {
        panic("pmem_alloc: out of memory");
    }

    // 清零后返回
    memset(page, 0, PGSIZE);
    return page;
}
```

1.3.3 pmem_free:释放物理页

回收一个物理页:

- 根据 `is_kernel` 选择对应的 `alloc_region`
- 检查参数 `page` (要释放的物理页起始地址) 的**合法性**
- **调试填充**: 使用 `memset` 将页面的每个字节都写成1, 便于后续发现野指针/重复释放错误。
- 获取 `alloc_region` 的锁, 将该页插入链表头 (**头插法**) ; 更新链表头和 `allocable` , 释放锁

```
// 释放一个物理页
// 失败则panic锁死
void pmem_free(uint64 page, bool in_kernel)
```

```

{
    // page: 要释放的物理页的起始地址
    // 释放一个之前申请的物理页:
    // 将它插入空闲链表的表头

    // 分配区域: 内核区域 or 用户区域
    alloc_region_t *ar = &user_region;
    if (in_kernel)
    {
        ar = &kern_region;
    }

    // 检查page的合法性
    if (page % PGSIZE != 0 || page < ar->begin || page >= ar->end)
    {
        panic("pmem_free: invalid page");
    }

    // 调试用: 填充为垃圾值
    memset((void *)page, 1, PGSIZE);

    // 插入到空闲链表的表头
    page_node_t *p = (page_node_t *)page;
    spinlock_acquire(&ar->lk);
    p->next = ar->list_head.next;
    ar->list_head.next = p;
    ar->allocable++;
    spinlock_release(&ar->lk);
}

```

1.4 测试用例

test-1

```

volatile static int started = 0;

volatile static int over_1 = 0, over_2 = 0;

static int* mem[1024];

int main()
{
    int cpuid = r_tp();

    if(cpuid == 0) {

        print_init();
        pmem_init();

        printf("cpu %d is booting!\n", cpuid);
        __sync_synchronize();
        started = 1;

        for(int i = 0; i < 512; i++) {
            mem[i] = pmem_alloc(true);
        }
    }
}

```

```

        memset(mem[i], 1, PGSIZE);
        printf("mem = %p, data = %d\n", mem[i], mem[i][0]);
    }
    printf("cpu %d alloc over\n", cpuid);
    over_1 = 1;

    while(over_1 == 0 || over_2 == 0);

    for(int i = 0; i < 512; i++)
        pmem_free((uint64)mem[i], true);
    printf("cpu %d free over\n", cpuid);

} else {

    while(started == 0);
    __sync_synchronize();
    printf("cpu %d is booting!\n", cpuid);

    for(int i = 512; i < 1024; i++) {
        mem[i] = pmem_alloc(true);
        memset(mem[i], 1, PGSIZE);
        printf("mem = %p, data = %d\n", mem[i], mem[i][0]);
    }
    printf("cpu %d alloc over\n", cpuid);
    over_2 = 1;

    while(over_1 == 0 || over_2 == 0);

    for(int i = 512; i < 1024; i++)
        pmem_free((uint64)mem[i], true);
    printf("cpu %d free over\n", cpuid);

}
while (1);
}

```

这个测试用例的作用是：

1. cpu-0和cpu-1并行申请内核空间的全部物理内存, 赋值并输出信息
2. 待申请全部结束, 并行释放所有申请的物理内存

输出结果如下：

```
mem = 80013000, data = 16843009
mem = 80012000, data = 16843009
mem = 80011000, data = 16843009
mem = 80010000, data = 16843009
cpu 1 alloc over
mem = 8000f000, data = 16843009
mem = 8000e000, data = 16843009
mem = 8000d000, data = 16843009
mem = 8000c000, data = 16843009
mem = 8000b000, data = 16843009
mem = 8000a000, data = 16843009
mem = 80009000, data = 16843009
mem = 80008000, data = 16843009
mem = 80007000, data = 16843009
mem = 80006000, data = 16843009
cpu 0 alloc over
cpu 0 free over
cpu 1 free over
[]
```

成功通过test_1!

test-2

```
// 测试目标：耗尽内核/用户区域内存
void test_case_1()
{
    void *page = NULL;

    while (1)
    {
        page = pmem_alloc(true);
        // page = pmem_alloc(false);
    }
}

#define TEST_CNT 10

// 测试目标：常规申请和释放操作
void test_case_2()
{
    alloc_region_t *user_ar = &user_region;
    uint64 user_pages[TEST_CNT];

    for (int i = 0; i < TEST_CNT; i++)
        user_pages[i] = 0;
```

```

printf("=== test_case_2: Phase 1 - Allocate User Pages ===\n");
for (int i = 0; i < TEST_CNT; i++)
{
    user_pages[i] = (uint64_t)pmem_alloc(false);

    printf("Allocated user page[%d] @ %p\n", i, (void *)user_pages[i]);

    if (!(user_pages[i] >= user_ar->begin && user_pages[i] < user_ar->end))
    {
        printf("Assertion failed: Page address out of bounds! Page: %p,
Region: [%p, %p)\n",
            (void *)user_pages[i], (void *)user_ar->begin, (void
*)user_ar->end);
        panic("Page address out of user region bounds");
    }

    memset((void *)user_pages[i], 0xAA, PGSIZE);
}

printf("=== test_case_2: Phase 2 - Pre-free Check ===\n");
spinlock_acquire(&user_ar->lk);
int expected_before = (user_ar->end - user_ar->begin) / PGSIZE - TEST_CNT;
int actual = user_ar->allocable;
printf("Expected allocable: %d, Actual: %d\n", expected_before, actual);
assert(user_ar->allocable == expected_before, "Allocable count incorrect
before free");
spinlock_release(&user_ar->lk);

printf("=== test_case_2: Phase 3 - Free Pages ===\n");
for (int i = 0; i < TEST_CNT; i++)
{
    pmem_free(user_pages[i], false);
    printf("Free user page[%d] @ %p\n", i, (void *)user_pages[i]);
}

printf("=== test_case_2: Phase 4 - Post-free Check ===\n");
spinlock_acquire(&user_ar->lk);
int expected_after = (user_ar->end - user_ar->begin) / PGSIZE;
actual = user_ar->allocable;
printf("Expected allocable: %d, Actual: %d\n", expected_after, actual);
assert(user_ar->allocable == expected_after, "Allocable count not restored
after free");
if (user_ar->list_head.next != NULL)
    printf("Free list head @ %p\n", user_ar->list_head.next);
else
    panic("Free list is empty after freeing pages");
spinlock_release(&user_ar->lk);

printf("=== test_case_2: Phase 5 - Reallocate & Verify Zero ===\n");
for (int i = 0; i < TEST_CNT; i++)
{
    void *page = pmem_alloc(false);
    printf("Reallocated page[%d] @ %p\n", i, page);

    bool non_zero = false;
    for (int j = 0; j < PGSIZE / sizeof(int); j++)

```



```

    {
        if (((int *)page)[j] != 0)
        {
            non_zero = true;
            printf("Non-zero value detected at offset %d: 0x%x\n", j, ((int
*)page)[j]);
            break;
        }
    }
    assert(!non_zero, "Memory not zeroed after free");
    printf("Zero verification passed\n");
}

printf("test_case_2 passed!\n");
}

```

这个测试用例的作用是：

1. 测试内存耗尽的 panic 是否正常触发
2. 测试用户空间物理页申请和释放的正确性

test_case_1时发现 panic 未能正常触发, 经过排查是因为 panic 逻辑有误: panic 里先设置 panicked=1, 再调用 printf; 而 printf 开头检测到如果 panicked==1 就直接返回, 因此看不到 panic 的输出信息。

修改 panic 函数: 将 panicked=1 放到 printf 之后。

```

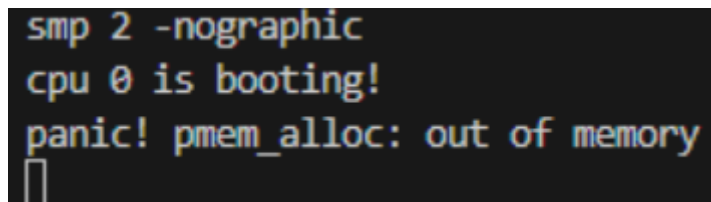
/* 报错并终止输出 */
void panic(const char *s)
{
    push_off(); //关中断

    if(!panicked){ //避免不同核重复调用
        printf("panic! %s\n", s?s:"<null>"); //先报错
        panicked = 1; //再锁死
    }

    while (1){
        asm volatile("wfi"); //安全停机
    }
}

```

test_case_1输出结果如下：



```

smp 2 -nographic
cpu 0 is booting!
panic! pmem_alloc: out of memory

```

成功通过test_case_1!

test_case_2输出结果如下：

```

cpu 0 is booting!
=== test_case_2: Phase 1 - Allocate User Pages ===

```

```
Allocated user page[0] @ 87fff000
Allocated user page[1] @ 87ffe000
Allocated user page[2] @ 87ffd000
Allocated user page[3] @ 87ffc000
Allocated user page[4] @ 87ffb000
Allocated user page[5] @ 87ffa000
Allocated user page[6] @ 87ff9000
Allocated user page[7] @ 87ff8000
Allocated user page[8] @ 87ff7000
Allocated user page[9] @ 87ff6000
=== test_case_2: Phase 2 - Pre-free Check ===
Expected allocable: 31730, Actual: 31730
=== test_case_2: Phase 3 - Free Pages ===
Free user page[0] @ 87fff000
Free user page[1] @ 87ffe000
Free user page[2] @ 87ffd000
Free user page[3] @ 87ffc000
Free user page[4] @ 87ffb000
Free user page[5] @ 87ffa000
Free user page[6] @ 87ff9000
Free user page[7] @ 87ff8000
Free user page[8] @ 87ff7000
Free user page[9] @ 87ff6000
=== test_case_2: Phase 4 - Post-free Check ===
Expected allocable: 31740, Actual: 31740
Free list head @ 87ff6000
=== test_case_2: Phase 5 - Reallocate & Verify Zero ===
Reallocated page[0] @ 87ff6000
Zero verification passed
Reallocated page[1] @ 87ff7000
Zero verification passed
Reallocated page[2] @ 87ff8000
Zero verification passed
Reallocated page[3] @ 87ff9000
Zero verification passed
Reallocated page[4] @ 87ffa000
Zero verification passed
Reallocated page[5] @ 87ffb000
Zero verification passed
Reallocated page[6] @ 87ffc000
Zero verification passed
Reallocated page[7] @ 87ffd000
Zero verification passed
Reallocated page[8] @ 87ffe000
Zero verification passed
Reallocated page[9] @ 87fff000
Zero verification passed
test_case_2 passed!
cpu 0 test over
```

成功通过test_case_2!

第二阶段：内核态虚拟内存

使用RISC-V体系结构中的SV39作为虚拟内存的设计规范：即39 位虚拟地址，三级页表。

2.1 相关概念与原理

- **虚拟地址 (VA) 结构:**
 - $VPN[2] + VPN[1] + VPN[0] + OFFSET$
 - 9bit 9bits 9bits 12bits (共39bits)
 - **VPN**为对应层级的**虚拟页号**，每一级的 `VPN[i]` 占 9 位，每级最多索引 512 项 (因为 $2^9 = 512$)
 - 在 `mem/type.h` 中定义的宏 `VA_TO_VPN(va, level)`：用于提取第 `level` 层的虚拟页号 (9 bits)
- **页表项 (PTE) 结构:**
 - reserved + PPN + RSW + D A G U X W R V
 - 10bits 44bits 2bits 8bits (共64bits)
 - **一个页表项对应一个物理页**，PPN为该页表项管理的物理页的页号，低10bits为它所管理的物理页的标志位
 - 页表本身也是存放在物理页中，这种物理页的特点是PTE的标志位中 `PTE_R PTE_W PTE_X` 都是 0，因此可以说，物理页被组织为：**存放页表的物理页&存放数据的物理页**
 - 在 `mem/type.h` 中定义的宏 `PTE_CHECK(pte)`：用于检查一个PTE是否属于页表 (是否是存放页表的物理页)
 - **关于页表和页表项:**

页表由页表项构成的，页表相当于数组，页表项相当于数组里的元素

```
// 页表项和页表(页表项数组)
typedef uint64 pte_t; //页表项pte_t : 64位无符号数
typedef pte_t* pgtbl_t; //页表pte_t* : 指向pte_t数组的指针
```

- **关于页表项和物理地址 (PA) :**
 - 物理地址 (PA) 结构位：PPN (44位有效) + OFFSET (12位)，其中offset为0，因为**页 (Page)** 是内存管理的基本单位，操作系统只按“整页”来分配和映射内存。所以所有被映射的物理地址都必须是 **4KB 对齐的**，即**低 12 位为 0** → offset = 0。
 - 页表项与物理地址的转换：页表项和物理地址中的**PPN相等**，不同点在于PA的**低12位**是 offset，PTE的**低10位**是标志位。在 `mem/type.h` 中定义了如下**页表项与物理地址的转换**的宏：

```
#define PA_TO_PTE(pa) (((uint64)(pa)) >> 12) << 10)
先直接去掉为0的12位offset 再左移10位给flag挪位置
#define PTE_TO_PA(pte) (((uint64)(pte) >> 10) << 12)
先去掉10位标志位再恢复12位offset
```

2.2 核心操作函数实现

在 `kvm.c` 中按照 `vm_getpte -> vm_mappages -> vm_unmappages` 的顺序实现三个核心的操作函数。

2.2.1 vm_getpte 函数

- 目的：根据 `pagetable` 找到 `va` 对应的 `pte`。
- 参数：
 - `pgtbl`：根页表指针（类型为 `pgtbl_t`，即 `pte_t*`）
 - `va`：uint64（64位无符号整数），即要查找的虚拟地址
 - `alloc`：bool类型，是否允许在路径缺失时自动分配中间页表

- 返回值：

若成功返回 `va` 对应的 `pte`，失败返回 `NULL`。

- 具体实现：

首先检查虚拟地址的合法性（与 `mem/type.h` 中定义的 `VA_MAX`）相比较

```
// 检查地址合法性
if (va >= VA_MAX)
    return NULL;
```

然后利用 `VA_TO_VPN` 来逐个提取当前层级的虚拟页号，并结合当前页表获取当前层级的PTE。遍历到 `level=0` 时直接返回当前PTE，遍历中间层时先检查获取的PTE是否有效：

- PTE有效：利用 `PTE_CHECK` 检查是否符合中间节点的要求（R/W/X均为0），若符合要求，则利用 `PTE_TO_PA` 讲当前层PTE设为下一级的页表首地址
- PTE无效：根据 `alloc` 参数判断是否需要自动分配中间页，若不允许分配，直接返回失败 `NULL`；若允许分配，则利用 `pmem_alloc(true)` 分配新一页作为下一级的页表，并利用 `PA_TO_PTE` 设置 PTE 指向新页表。

具体代码如下：

```
pgtbl_t curr = pgtbl; // 当前正在查看的页表

// level=2 和 level=1（中间层）
for (int level = 2; level > 0; level--) {
    int idx = VA_TO_VPN(va, level); // 获取当前层级虚拟页号
    pte_t *pte = &curr[idx];        // 当前层级的 PTE

    if (*pte & PTE_V) {               // PTE 有效
        if (PTE_CHECK(*pte)) {        // 是中间节点（R/W/X=0）
            uint64 child_pa = PTE_TO_PA(*pte);
            curr = (pgtbl_t)child_pa; // 跳转到下一级页表
        } else {
            return NULL; // 非法：中间节点设置了 R/W/X
        }
    } else {                          // PTE 无效
        // 不允许分配
        if (!alloc)
            return NULL;

        // 允许分配
        void *pa = pmem_alloc(true); // 分配一页作为下一级页表
        if (!pa) return NULL;
        memset(pa, 0, PGSIZE);       // 清零

        uint64 child_ppn = PA_TO_PTE((uint64)pa);
```

```

        *pte = child_ppn | PTE_V;           // 设置 PTE 指向新页表

        curr = (pgtbl_t)pa;                 // 更新当前页表为新分配的
    }
}

// 到达 level=0
// 直接返回 level=0 的 PTE 指针
int idx = VA_TO_VPN(va, 0);
return &curr[idx];

```

2.2.4 vm_mappages 函数

- 目的：在页表 pgtbl 中建立 `[va, va + len)` -> `[pa, pa + len)` 的映射
- 参数：
 - pgtbl：根页表指针（类型为 pgtbl_t，即 pte_t*）
 - va：uint64（64位无符号整数），虚拟地址映射区间首地址
 - pa：uint64（64位无符号整数），物理地址映射区间首地址
 - len：uint64（64位无符号整数），要建立映射的区间的长度
 - perm：int类型，构造的新的PTE的权限标志（如可读、可写、可执行等）
- 无返回值
- 具体实现：
 - 首先，要进行参数检查，主要检查三点：
 - len（长度）必须大于 0
 - va 和 pa 必须页对齐
 - va+len 后不能越界

```

//参数检查
// 1. 长度必须大于 0
if (len == 0) {
    panic("vm_mappages: len is zero");
}

// 2. va 和 pa 必须页对齐
if (va % PGSIZE != 0 || pa % PGSIZE != 0) {
    panic("vm_mappages: va or pa not page-aligned");
}

// 3. 不能越界
if (va + len > VA_MAX) {
    panic("vm_mappages: virtual address overflow");
}

```

- 接着，进行逐页映射（已 PGSIZE 整页为单位进行逐页映射）

建立映射的本质是找到 va 在页表对应位置的 pte 并修改它，即给对应的 pte 设置正确的物理页号（PPN），权限（perm）和有效位（V=1）

```

//逐页映射

```

```

uint64 end = va + len; // 结束虚拟地址

while (va < end) {
    // Step 1: 获取当前虚拟地址对应的 PTE 指针
    //          如果路径不存在, 自动创建中间页表)
    pte_t *pte = vm_getpte(pgtbl, va, true);
    if (!pte) {
        panic("vm_mappages: cannot create PTE (out of memory?)");
    }

    // Step 2: 修改 PTE
    //          将物理地址 pa 编码为 PPN 字段, 并加上权限和 V 标志
    uint64 pte_flags = PA_TO_PTE(pa) | perm | PTE_V;
    *pte = pte_flags;

    // Step 3: 前进到下一页
    va += PGSIZE;
    pa += PGSIZE;
}

```

2.2.3 vm_unmappages 函数

- 目的: 在页表 `pgtbl` 中解除虚拟地址区间 `[va, va + len)` 的映射并释放资源
- 参数:
 - `pgtbl`: 根页表指针 (类型为 `pgtbl_t`, 即 `pte_t*`)
 - `va`: `uint64` (64位无符号整数), 要解除的虚拟地址映射区间首地址
 - `len`: `uint64` (64位无符号整数), 要解除的虚拟地址映射区间长度
 - `freeit`: `bool`类型, 是否释放对应的物理资源, 如果 `freeit` = `true`则释放对应物理页, 默认是用户的物理页
- 无返回值
- 具体实现:
 - 首先, 与 `vm_mappages` 函数一样, 先进行参数检查

```

//参数检查
// 1. 长度必须大于 0
if (len == 0) {
    panic("vm_unmappages: len is zero");
}
// 2. va 必须页对齐
else if (va % PGSIZE != 0) {
    panic("vm_unmappages: va not page-aligned");
}
// 3. 不能越界
else if (va + len > VA_MAX) {
    panic("vm_unmappages: virtual address overflow");
}

```

- 然后, 逐页解除映射, 先获取当前虚拟地址对应的 PTE 指针(不允许自动创建中间页表), 然后将该PTE标记为无效; 如果需要, 还要释放对应的物理页 (默认是用户的物理页)。

```

//逐页解除映射
uint64 end = va + len; // 结束虚拟地址

```

```

while (va < end) {
    // Step 1: 获取当前虚拟地址对应的 PTE 指针(不允许自动创建)
    pte_t *pte = vm_getpte(pgtbl, va, false);
    if (!pte || !(*pte & PTE_V)) {
        panic("vm_unmappages: unmap a not mapped page");
    }

    // Step 2: 如果需要, 释放对应的物理页
    if (freeit) {
        uint64 pa = PTE_TO_PA(*pte);

        // 释放 默认是用户的物理页
        pmem_free(pa, false);
    }

    // Step 3: 将 PTE 标记为无效 (解除映射)
    *pte = 0;

    // Step 4: 前进到下一页
    va += PGSIZE;
}

```

2.3 设置内核页表映射并启用

完成三个核心的页表操作函数后, 需要给内核页表 `kernel_pgtbl` 设置映射关系并为每个CPU启用它, 对应的函数在: `kvm_init` -> `kvm_inithart`

2.3.1 `kvm_init` 函数

- 目的: 完成UART、CLINT、PLIC、内核代码区、内核数据区、可分配区域的页表映射
- 具体实现:
 - 首先, **分配根页表** (即第2级页表)。从内核区域分配一页并将该页清零

```

kernel_pgtbl = (pgtbl_t)pmem_alloc(true); // 从内核区域分配一页
if (!kernel_pgtbl) {
    panic("kvm_init: cannot allocate root page table");
}
memset(kernel_pgtbl, 0, PGSIZE); // 清零

```

- 然后, **映射内核代码和数据区**。首先获取内核代码和数据的范围, 然后利用上面写的 `vm_mappages` 函数建立映射, 为恒等映射 (`pa = va`)

```

// === 获取内核代码和数据的范围 ===
uint64 text_start = KERNEL_BASE; // 代码段开始的位置
uint64 data_end   = (uint64)ALLOC_BEGIN; // 数据段结束位置
uint64 size = data_end - text_start; // 长度
uint64 map_size = (size + PGSIZE - 1) & ~(PGSIZE - 1); // 对齐为整页

// 映射到内核代码和数据区===
vm_mappages(kernel_pgtbl, text_start, text_start, map_size, PTE_R | PTE_W |
PTE_X); // pa=va 恒等映射

```

- 接着，对 UART/CLINT/PLIC 等设备的寄存器进行映射，同样是利用上面写的 `vm_mappages` 函数建立恒等映射。（并且由于源码没有这些设备的位置，就先在 `kvm.c` 中人为define了这些设备的物理地址，不知道对不对，，）

```
// 设定设备的物理地址(函数外)
#define UART_ADDR      0x10000000ULL
#define CLINT_ADDR     0x02000000ULL
#define PLIC_ADDR      0x0C000000ULL

// 建立映射
vm_mappages(kernel_pgtbl, UART_ADDR, UART_ADDR, PGSIZE, PTE_R | PTE_W); // 不可执行

vm_mappages(kernel_pgtbl, CLINT_ADDR, CLINT_ADDR, 0x10000, PTE_R | PTE_W);
// 不可执行

vm_mappages(kernel_pgtbl, PLIC_ADDR, PLIC_ADDR, 0x4000000, PTE_R | PTE_W);
// 不可执行
```

- 最后，映射可用内存区域 `[ALLOC_BEGIN, ALLOC_END)`，为了防止未来成为攻击者注入代码的目标，设定为不可执行

```
uint64 phy_pool_begin = (uint64)ALLOC_BEGIN;
uint64 phy_pool_end   = (uint64)ALLOC_END;
uint64 phy_pool_sz    = phy_pool_end - phy_pool_begin;

vm_mappages(kernel_pgtbl, phy_pool_begin, phy_pool_begin, phy_pool_sz, PTE_R
| PTE_W); // 不可执行
```

2.3.2 kvm_inithart 函数

这是用于在每个 CPU 核心（hart）上初始化虚拟内存系统的函数，即启用分页机制，切换到内核页表。

◦ 关于 `satp` 寄存器：

- 作用：告诉 CPU 当前使用的页表根地址在哪里，以及是否启用虚拟内存（分页机制）
- 结构：
 - MODE：4bit-地址翻译模式（如8=SV39）
 - ASID：16bit-地址空间ID（可加快TLB切换）
 - PPN：44bit-页表根节点的物理页号
- 在 `mem/type.h` 中定义的宏，可设置 `satp` 寄存器的MODE和PPN字段

```
#define SATP_SV39 (8L << 60) // MODE = SV39
#define MAKE_SATP(pagetable) (SATP_SV39 | (((uint64)pagetable) >> 12))
// 设置MODE和PPN字段
```

◦ 函数流程

- 写入 `satp` 寄存器
- 刷新TLB缓存


```

void kvm_inithart()
{
    w_satp(MAKE_SATP(kernel_pgtbl)); //写入 satp 寄存器
    sfence_vma(); // 刷新TLB缓存
}

```

也就是说，调用了这个函数后，就启用了内核页表，可以开始使用虚拟地址了！

2.4 测试用例

test-1

```

int main()
{
    int cpuid = r_tp();

    if(cpuid == 0) {

        print_init();
        pmem_init();
        kvm_init();
        kvm_inithart();

        printf("cpu %d is booting!\n", cpuid);
        __sync_synchronize();
        // started = 1;

        pgtbl_t test_pgtbl = pmem_alloc(true);
        uint64 mem[5];
        for(int i = 0; i < 5; i++)
            mem[i] = (uint64)pmem_alloc(false);

        printf("\ntest-1\n\n");
        vm_mappages(test_pgtbl, 0, mem[0], PGSIZE, PTE_R);
        vm_mappages(test_pgtbl, PGSIZE * 10, mem[1], PGSIZE / 2, PTE_R |
PTE_W);
        vm_mappages(test_pgtbl, PGSIZE * 512, mem[2], PGSIZE - 1, PTE_R |
PTE_X);
        vm_mappages(test_pgtbl, PGSIZE * 512 * 512, mem[2], PGSIZE, PTE_R |
PTE_X);
        vm_mappages(test_pgtbl, VA_MAX - PGSIZE, mem[4], PGSIZE, PTE_W);
        vm_print(test_pgtbl);

        printf("\ntest-2\n\n");
        vm_mappages(test_pgtbl, 0, mem[0], PGSIZE, PTE_W);
        vm_unmappages(test_pgtbl, PGSIZE * 10, PGSIZE, true);
        vm_unmappages(test_pgtbl, PGSIZE * 512, PGSIZE, true);
        vm_print(test_pgtbl);

    } else {

        while(started == 0);
        __sync_synchronize();
        printf("cpu %d is booting!\n", cpuid);
    }
}

```

```

    }
    while (1);
}

```

这个测试用例测试了两件事情:

1. 使用内核页表后OS内核是否还能**正常执行**
2. 使用映射和解映射操作**修改你的页表**, 使用vm_print输出它被修改前后的对比

测试的输出结果如下:

```

qemu-system-riscv64 -machine virt -bios none -kernel target/kernel/kernel-qemu.elf
-m 128M -smp 2 -nographic
cpu 0 is booting!

test-1

level-2 pgtbl: pa = 8039f000
.. level-1 pgtbl 0: pa = 8039e000
.. .. level-0 pgtbl 0: pa = 8039d000
.. .. . physical page 0: pa = 87fff000 flags = 3
.. .. . physical page 10: pa = 87ffe000 flags = 7
.. .. level-0 pgtbl 1: pa = 8039c000
.. .. . physical page 0: pa = 87ffd000 flags = 11
.. level-1 pgtbl 1: pa = 8039b000
.. .. level-0 pgtbl 0: pa = 8039a000
.. .. . physical page 0: pa = 87ffd000 flags = 11
.. level-1 pgtbl 255: pa = 80399000
.. .. level-0 pgtbl 511: pa = 80398000
.. .. . physical page 511: pa = 87ffb000 flags = 5

test-2

level-2 pgtbl: pa = 8039f000
.. level-1 pgtbl 0: pa = 8039e000
.. .. level-0 pgtbl 0: pa = 8039d000
.. .. . physical page 0: pa = 87fff000 flags = 5
.. .. level-0 pgtbl 1: pa = 8039c000
.. level-1 pgtbl 1: pa = 8039b000
.. .. level-0 pgtbl 0: pa = 8039a000
.. .. . physical page 0: pa = 87ffd000 flags = 11
.. level-1 pgtbl 255: pa = 80399000
.. .. level-0 pgtbl 511: pa = 80398000
.. .. . physical page 511: pa = 87ffb000 flags = 5
QEMU: Terminated
wxc@wxc-virtual-machine:~/Work/ANOS$

```

test-2

```

/*----- 测试代码 -----*/

void test_mapping_and_unmapping()
{
    // 1. 初始化测试页表
    pte_t* pte;
    pgtbl_t pgtbl = (pgtbl_t)pmem_alloc(true);
    memset(pgtbl, 0, PGSIZE);

    // 2. 准备测试条件
    uint64 va_1 = 0x100000;
    uint64 va_2 = 0x8000;
}

```

```

uint64 pa_1 = (uint64)pmem_alloc(false);
uint64 pa_2 = (uint64)pmem_alloc(false);

// 3. 建立映射
vm_mappages(pgtbl, va_1, pa_1, PGSIZE, PTE_R | PTE_W);
vm_mappages(pgtbl, va_2, pa_2, PGSIZE, PTE_R);

// 4. 验证映射结果
pte = vm_getpte(pgtbl, va_1, false);
assert(pte != NULL, "test_mapping_and_unmapping: pte_1 not found");
assert((*pte & PTE_V) != 0, "test_mapping_and_unmapping: pte_1 not valid");
assert(PTE_TO_PA(*pte) == pa_1, "test_mapping_and_unmapping: pa_1 mismatch");
assert((*pte & (PTE_R | PTE_W)) == (PTE_R | PTE_W),
"test_mapping_and_unmapping: flag_1 mismatch");

pte = vm_getpte(pgtbl, va_2, false);
assert(pte != NULL, "test_mapping_and_unmapping: pte_2 not found");
assert((*pte & PTE_V) != 0, "test_mapping_and_unmapping: pte_2 not valid");
assert(PTE_TO_PA(*pte) == pa_2, "test_mapping_and_unmapping: pa_2 mismatch");
assert((*pte & (PTE_R | PTE_W)) == (PTE_R | PTE_W),
"test_mapping_and_unmapping: flag_2 mismatch");

// 5. 解除映射
vm_unmappages(pgtbl, va_1, PGSIZE, true);
vm_unmappages(pgtbl, va_2, PGSIZE, true);

// 6. 验证解除映射结果
pte = vm_getpte(pgtbl, va_1, false);
assert(pte != NULL, "test_mapping_and_unmapping: pte_1 not found");
assert((*pte & PTE_V) == 0, "test_mapping_and_unmapping: pte_1 still valid");
pte = vm_getpte(pgtbl, va_2, false);
assert(pte != NULL, "test_mapping_and_unmapping: pte_2 not found");
assert((*pte & PTE_V) == 0, "test_mapping_and_unmapping: pte_2 still valid");

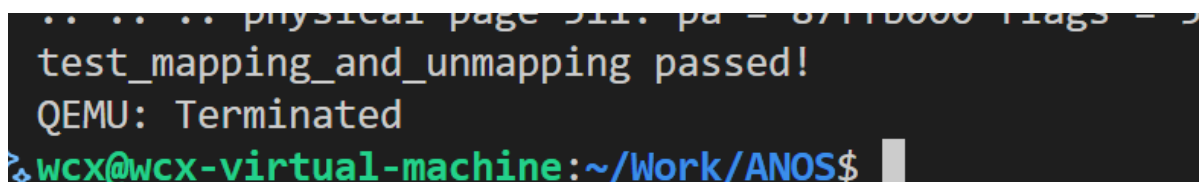
// 7. 由于页表的释放函数还没实现，作为测试用例可以展示不释放页表空间

printf("test_mapping_and_unmapping passed!\n");
}

```

这个测试用例主要关注映射和解映射是否正确执行

在 main.c 中调用 test_mapping_and_unmapping() 进行测试，输出：test_mapping_and_unmapping passed!，表示通过测试，截图如下：



```

.. .. .. physical page 511: pa = 07110000 flags = 5
test_mapping_and_unmapping passed!
QEMU: Terminated
wcx@wcx-virtual-machine:~/Work/ANOS$

```

补充测试用例

为了进一步增强代码的稳定性，增加了一些内存映射的边缘测试用例，测试函数如下：

```

void test_vm_edge_cases()
{

```

```

pgtbl_t pgtbl = (pgtbl_t)pmem_alloc(true);
memset(pgtbl, 0, PGSIZE);

uint64 pa1 = (uint64)pmem_alloc(false);
uint64 pa2 = (uint64)pmem_alloc(false);

pte_t *pte;

// Test 1: 映射 VA=0
vm_mappages(pgtbl, 0, pa1, PGSIZE, PTE_R);
pte = vm_getpte(pgtbl, 0, false);
assert(pte && (*pte & PTE_V), "test_vm_edge_cases: mapping va=0 failed");
assert(PTE_TO_PA(*pte) == pa1, "test_vm_edge_cases: pa mismatch at va=0");

// Test 2: 映射接近 VA_MAX 的地址
uint64 high_va = VA_MAX - PGSIZE;
vm_mappages(pgtbl, high_va, pa2, PGSIZE, PTE_X);
pte = vm_getpte(pgtbl, high_va, false);
assert(pte && (*pte & PTE_V), "test_vm_edge_cases: high va mapping failed");
assert(PTE_TO_PA(*pte) == pa2, "test_vm_edge_cases: pa mismatch at high va");

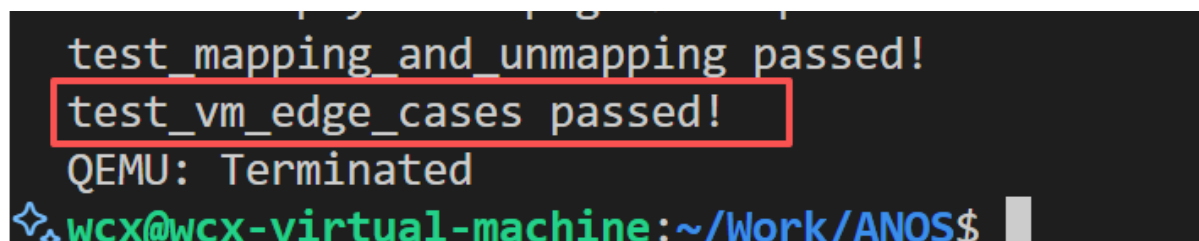
// Test 3: 解除映射并验证
vm_unmappages(pgtbl, 0, PGSIZE, true);
pte = vm_getpte(pgtbl, 0, false);
assert(pte && !(*pte & PTE_V), "test_vm_edge_cases: unmap failed for va=0");

// Test 4: 尝试 remap 到同一 VA (更新权限)
uint64 pa3 = (uint64)pmem_alloc(false);
vm_mappages(pgtbl, 0, pa3, PGSIZE, PTE_W | PTE_X);
pte = vm_getpte(pgtbl, 0, false);
assert((*pte & (PTE_W | PTE_X)) == (PTE_W | PTE_X),
       "test_vm_edge_cases: permission not set correctly");

printf("test_vm_edge_cases passed!\n");
}

```

最终打印出 `test_vm_edge_cases passed!`，表示通过了补充的边缘测试！



A terminal window screenshot with a black background and white text. The output shows three lines: 'test_mapping_and_unmapping passed!', 'test_vm_edge_cases passed!' (which is highlighted with a red rectangular box), and 'QEMU: Terminated'. Below this, the prompt '✧ wcx@wcx-virtual-machine:~/Work/ANOS\$' is visible in green and white text.